

Lab 4 — Balloon Pop

LLM Prompts — Tasks 1 to 4 only

Prompt 1 — Task 1: Fix Click Detection

I am working on the Balloon Pop Lab project.

Please inspect the existing code in:

- main.py
- game/game_engine.py
- game/balloon.py
- game/click_detection.py
- game/renderer.py

Do not rewrite the project or change unrelated functionality.

Task 1:

Fix the click-detection bug in game/click_detection.py.

The current check_pop function calculates the squared distance from the mouse click to the balloon's center, but compares that squared distance directly against the balloon's normal radius.

The intended behaviour is that a click anywhere within the visible circular balloon should pop it, not only a click very close to the centre.

First explain the existing bug and why the current comparison is incorrect. Then make the minimal code change required to fix Task 1.

Keep the existing project structure and do not add unnecessary dependencies.

Prompt 2 — Task 2: Balloon Types

Now implement Task 2 only.

Add at least three balloon types to the existing Balloon Pop game:

1. Normal balloon – awards regular points.
2. Bonus balloon – awards more points.
3. Penalty balloon – reduces the score or applies another clearly defined negative effect.

Requirements:

- Each balloon type must have a clearly different colour.
- The correct score/effect must occur when each type is popped.
- Keep the existing game architecture and modules.
- Do not rewrite unrelated functionality.
- Do not add unnecessary dependencies.
- Keep the Task 1 click-detection fix working.

First explain how balloon types should fit into the existing Balloon class and game logic.

Then make the minimal changes required.

Prompt 3 — Task 3: Lives/Miss System

Now implement Task 3 only.

Add a lives/miss system to the existing Balloon Pop game.

Requirements:

- Start the player with exactly 3 lives.
- Whenever a balloon reaches the bottom of the screen without being popped, decrease the life count by 1.
- Display the remaining lives clearly on screen.
- End the game when lives reach zero.
- Popping a balloon must never reduce a life, regardless of balloon type.
- Keep Task 1 click detection working.
- Keep Task 2 balloon types and their effects working.
- Do not rewrite unrelated functionality.
- Do not add unnecessary dependencies.

First explain where the life state and missed-balloon handling should be implemented. Then make the minimal code changes required.

Prompt 4 — Task 4: 30-Second Timed Round

Now implement Task 4 only.

Add a 30-second countdown round to the existing Balloon Pop game.

Requirements:

- Start each round with 30 seconds.
- Display the remaining time clearly on screen.
- Count the timer down during gameplay.
- When the timer reaches zero, stop spawning and accepting balloons.
- The round must also end if lives reach zero from Task 3.
- Show the final score clearly when the round ends.
- Provide a way to start a new round.
- Starting a new round must reset:
 - score
 - lives to 3
 - timer to 30 seconds
 - game state
- Keep Tasks 1, 2 and 3 working.
- Do not rewrite unrelated functionality.
- Do not add unnecessary dependencies.

First explain how the timer and round lifecycle should fit into the existing game engine.

Then make the minimal code changes required.

Use these four prompts sequentially and retain the complete LLM conversation for the Lab 4 submission.